

6DoF Flight Altitude Tracker Playbook

I. Challenge

Develop a real-time digital avionics instrument deck that measures dynamic physical orientation using a 6-Degrees-of-Freedom (6-DoF) Inertial Measurement Unit (IMU) and projects a responsive, fluid artificial horizon onto a dedicated graphical interface. The system must process dual high-speed hardware communication buses simultaneously while balancing sensor noise against calculation lag to provide lag-free flight telemetry.

II. Guide

A. Concept

To implement a high-refresh avionics dashboard, we leverage these fundamental embedded and mathematical principles:

1. **The 6-DoF Inertial Measurement Unit:** A sensor housing a 3-axis accelerometer (measuring linear acceleration due to gravity) and a 3-axis gyroscope (measuring angular rotational velocity). Combining these six vectors tracks orientation across pitch, roll, and yaw.

- Learn more: [DigiKey DIY Avionics Simulator Guide](#)

2. **Sensor Fusion & The Complementary Filter:** Accelerometers offer stable data long-term but suffer from high-frequency structural vibration noise. Gyroscopes track rapid movements perfectly but gather minor mathematical errors over time, causing "integration drift". A complementary filter resolves this by applying a weighted blending ratio to balance both data streams:

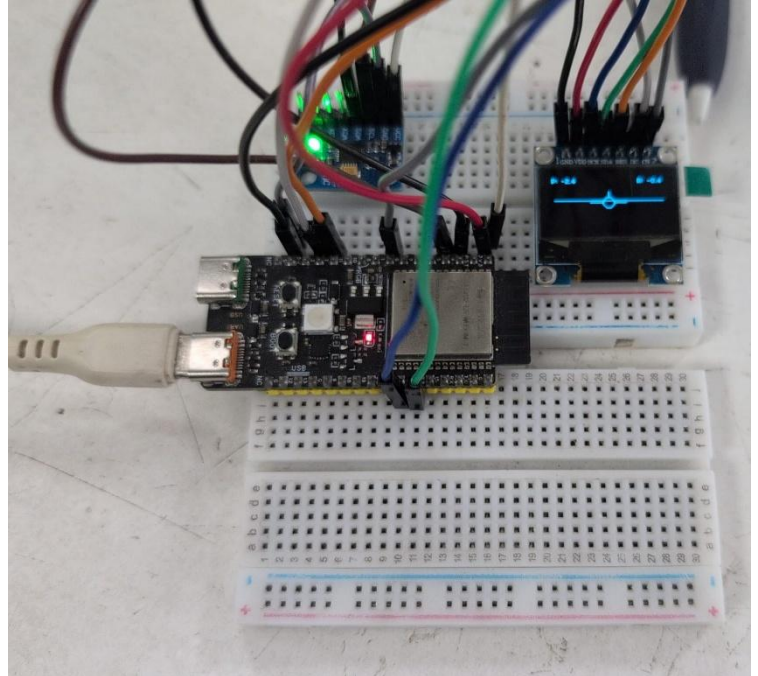
$$\theta_{\text{filtered}} = \alpha \cdot (\theta_{\text{prev}} + \omega \cdot dt) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

- Learn more: [Understanding Sensor Fusion](#)

- Learn more: [The Complementary Filter](#)

3. **2D Rotational Screen Projection:** Translating raw physical angles onto a flat, 2D display grid requires trigonometry. Changes in pitch shift the display center point up and down (Y_{offset}), while changes in roll rotate the horizon line's outer coordinates around that shifting pivot using sine and cosine functions.

- Learn more: [Working with OLED Displays](#)



4. **Dual-Bus Hardware Pipeline:** The microcontroller coordinates data over two separate communication interfaces concurrently: an I^2C bus configured in Fast Mode (400 kHz) to pull raw spatial arrays from the IMU, and an SPI bus (10 MHz) to push display updates.
- Learn more: [Communication Protocols](#)
5. **Full-Frame Memory Buffering:** Modifying individual pixels directly on a physical display causes screen tearing and visual jitter. Instead, the display driver sets aside a local 1024-byte array in the microcontroller's RAM to handle the calculation math secretly, flashing the completed image to the screen all at once via a single command transmission.
- Learn more: [U8g2 Graphic Engine Memory Modes](#)

Real-World Example: Glass cockpit Primary Flight Displays (PFD) inside modern commercial airliners (like the Boeing 787 or Airbus A350) and glass instrument panels in general aviation aircraft (like the Garmin G1000). They ingest data from highly sensitive solid-state micro-machined sensors and instantaneously project pitch and bank horizons to pilots.

B. Components

- **Microcontroller:** WeAct Studio ESP32-C6 (32-bit RISC-V SoC with an internal hardware Floating Point Unit).
- **Display:** 0.96-inch Simplex SPI monochrome OLED display module (128 × 64 resolution, SSD1306 driver).
- **Spatial Sensor:** MPU6050 6-DoF Integrated Circuit Module.
- **Prototyping Tools:** Solid hookup wires and a standard breadboard.

C. Connections

To keep the microcontroller stable during boot-up, communication lines are routed to clean general-purpose I/O pins, leaving system strapping pins completely untouched:

Peripheral Module	Physical Pin	Target Pin Designation	Signal Role	Electrical Conditions
SPI OLED	GND	GND	Ground Reference	0V Ground Plane
	VDD	3V3	Logic Power	3.3V Low-Noise Power Rail
	SCK	GPIO 21	FSPICLK	Master Clock Output (10 MHz)

Peripheral Module	Physical Pin	Target Pin Designation	Signal Role	Electrical Conditions
	SDA	GPIO 22	FSPID / MOSI	Master-Out Slave-In Data Interface
	RES	GPIO 2	GPIO Out	Active-Low Chip Reset Pin
	DC	GPIO 3	GPIO Out	Mode Switch (0 = Command, 1 = Data)
	CS	GPIO 1	FSPICS0	Active-Low Peripheral Selection
I2C IMU	VCC	3V3	Logic Power	Shared 3.3V Power Rail
	GND	GND	Ground Reference	Shared Ground Plane
	SCL	GPIO 5	I2C_SCL	Hardware I2C Clock (400 kHz)
	SDA	GPIO 4	I2C_SDA	Hardware I2C Data Link

D. Code

The program initializes the independent communication buses, calibrates the sensor registers, and establishes a filtered runtime loop to drive the graphics:

```
1. #include <Arduino.h>
2. #include <Wire.h>
3. #include <SPI.h>
4. #include <U8g2lib.h>
5. #include <Adafruit_MPU6050.h>
6. #include <Adafruit_Sensor.h>
7.
8. #define SPI_OLED_SCK  21
9. #define SPI_OLED_SDA  22
10. #define SPI_OLED_RES  2
11. #define SPI_OLED_DC   3
12. #define SPI_OLED_CS   1
13.
14. #define I2C_IMU_SDA    4
15. #define I2C_IMU_SCL    5
16.
17. U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI oled(U8G2_R0, SPI_OLED_CS, SPI_OLED_DC, SPI_OLED_RES);
18. Adafruit_MPU6050 mpu;
```

```

19.
20. float pitch = 0.0f;
21. float roll = 0.0f;
22. const float alpha = 0.96f;
23. uint32_t last_time = 0;
24.
25. void setup() {
26.     Serial.begin(115200);
27.     delay(1000);
28.
29.     // Initialize Independent Hardware Buses
30.     Wire.begin(I2C_IMU_SDA, I2C_IMU_SCL, 400000U); // Set I2C Fast Mode Speed to 400kHz
31.     SPI.begin(SPI_OLED_SCK, -1, SPI_OLED_SDA, SPI_OLED_CS);
32.
33.     if (!mpu.begin()) {
34.         Serial.println("[ERROR] MPU6050 Handshake Fault. System Halted.");
35.         while (1) { delay(10); }
36.     }
37.
38.     mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
39.     mpu.setGyroRange(MPU6050_RANGE_250_DEG);
40.     mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
41.
42.     oled.begin();
43.     last_time = micros();
44. }
45.
46. void loop() {
47.     uint32_t current_time = micros();
48.     float dt = (float)(current_time - last_time) / 1000000.0f;
49.     last_time = current_time;
50.
51.     if (dt <= 0.0f || dt > 0.5f) dt = 0.01f;
52.
53.     sensors_event_t a, g, temp;
54.     mpu.getEvent(&a, &g, &temp);
55.
56.     // Calculate accelerometer tilt vectors
57.     float accel_pitch = atan2(-a.acceleration.x, sqrt(a.acceleration.y * a.acceleration.y + a.acceleration.z *
a.acceleration.z)) * RAD_TO_DEG;
58.     float accel_roll = atan2(a.acceleration.y, a.acceleration.z) * RAD_TO_DEG;
59.
60.     // Convert angular velocities to degrees per second
61.     float gyro_pitch_rate = g.gyro.y * RAD_TO_DEG;
62.     float gyro_roll_rate = g.gyro.x * RAD_TO_DEG;
63.
64.     // Run Complementary Filter equations
65.     pitch = alpha * (pitch + (gyro_pitch_rate * dt)) + (1.0f - alpha) * accel_pitch;
66.     roll = alpha * (roll + (gyro_roll_rate * dt)) + (1.0f - alpha) * accel_roll;
67.
68.     oled.clearBuffer();
69.
70.     // Draw Static Center Crosshair Aircraft Indicator
71.     oled.setDrawColor(1);
72.     oled.drawCircle(64, 32, 5);
73.     oled.drawLine(46, 32, 12);
74.     oled.drawLine(70, 32, 12);
75.
76.     // Compute 2D Rotational Projection Mapping
77.     float rad_roll = (roll * DEG_TO_RAD);
78.     int y_offset = 32 + (int)(pitch * 1.2f); // Pitch displacement scalar factor
79.     int span_length = 55;
80.
81.     int x1 = 64 - (int)(span_length * cos(rad_roll));
82.     int y1 = y_offset - (int)(span_length * sin(rad_roll));
83.     int x2 = 64 + (int)(span_length * cos(rad_roll));
84.     int y2 = y_offset + (int)(span_length * sin(rad_roll));
85.
86.     // Render Dynamic Artificial Horizon Line
87.     oled.drawLine(x1, y1, x2, y2);
88.
89.     // Display Raw Data Output Fields

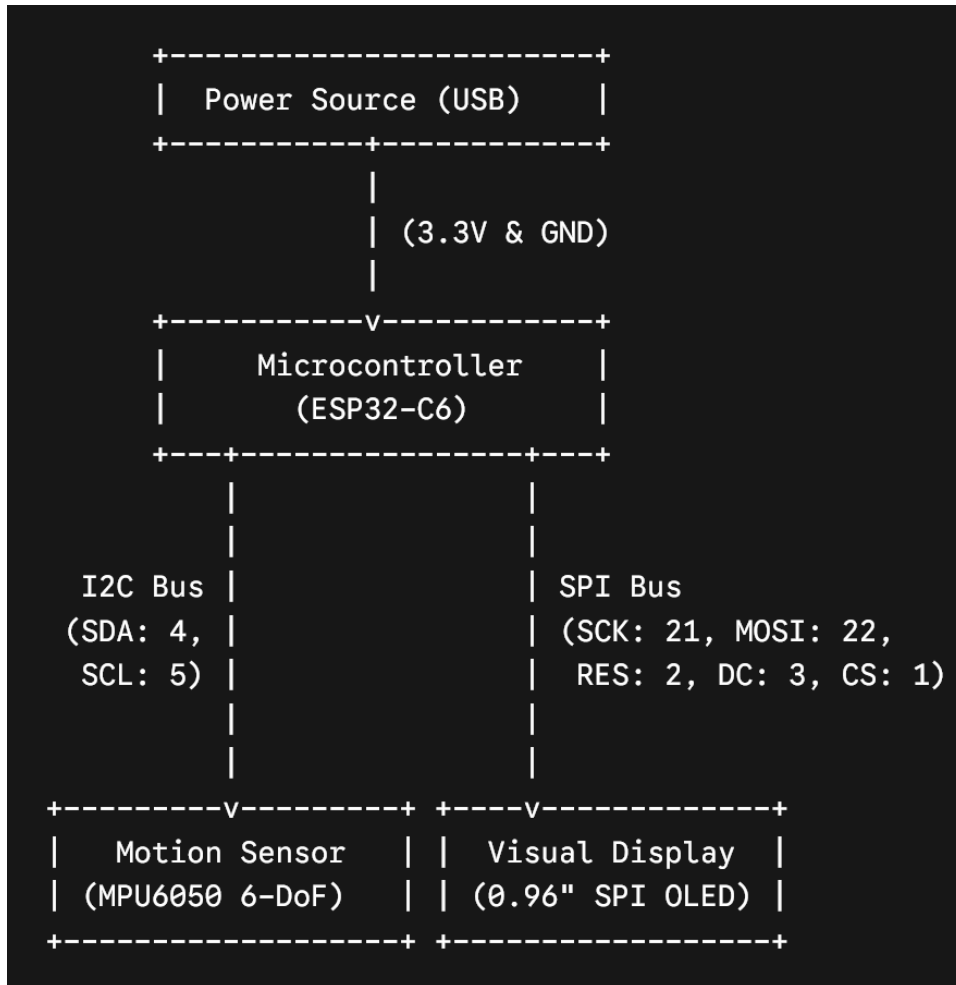
```

```

90. oled.setFont(u8g2_font_04b_03_tr);
91. oled.setCursor(2, 10); oled.print("PITCH: "); oled.print(pitch, 1);
92. oled.setCursor(88, 10); oled.print("ROLL: "); oled.print(roll, 1);
93.
94. oled.sendBuffer();
95. delay(10);
96. }
97.

```

E. Block Diagram



III. Play & Share

- **Upgrade to Kalman Filtering:** The Complementary Filter uses a fixed weighting factor (α). Can you replace it with a **Kalman Filter** algorithm? Explore how dynamically calculating measurement uncertainties updates your horizon tracking during sudden, erratic movements.
- **Add a Slip/Skid Indicator:** Real aircraft cockpits use a turning coordinator ball to ensure clean turns. Try drawing a small, dynamic curved tube at the bottom of the OLED screen and use the accelerometer's lateral Y-axis data to slide a physical indicator dot back and forth.
- **Build an Altitude Warning System:** Program a target altitude ceiling into your code. Connect an external active LED or a buzzer to a spare GPIO pin, and trigger a flashing hazard alert if the pitch angle exceeds a critical safety threshold.

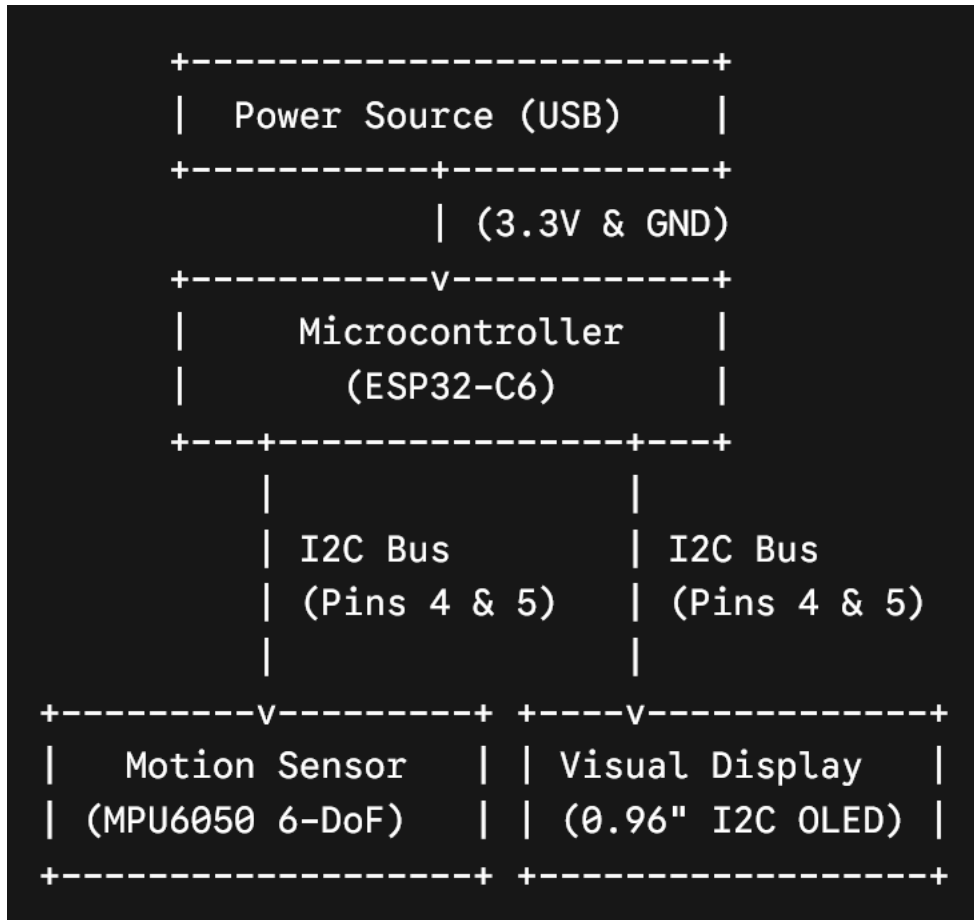
Use this section in case of parts variation

Hello! This section is designed to assist in the case of components variance. In simpler terms, did you get a component that has the same functions but the connections are different?

Some examples include:

- Your OLED Display has 4 pins instead of 7 pins? It turns out you got an [I2C Version!](#)
- Your buzzer module is either 2 pins or 3 pins

For our Aircraft Altimeter, we will now work with the I2C OLED which has 4 pins. Here is the Circuit Block Diagram:



Connections

Because both the display and the sensor use the I2C protocol, they will share the exact same Data (SDA) and Clock (SCL) pins on the ESP32-C6.

Component Pin	ESP32-C6 Pin	Description
Sensor GND & OLED GND	GND	The common ground connection (Minus).

Component Pin	ESP32-C6 Pin	Description
Sensor VCC & OLED VCC	3.3V	Provides clean logic power to turn both modules on.
Sensor SDA & OLED SDA	GPIO 4	The shared Data line for sending information back and forth.
Sensor SCL & OLED SCL	GPIO 5	The shared Clock line to keep communication timed correctly.

Code

```

1. #include <Arduino.h>
2. #include <Wire.h>
3. #include <U8g2lib.h>
4. #include <Adafruit_MPU6050.h>
5. #include <Adafruit_Sensor.h>
6.
7. // --- Shared I2C Pin Configuration ---
8. #define I2C_SDA 4
9. #define I2C_SCL 5
10.
11. // --- Initialize the I2C OLED Display and Sensor ---
12. U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);
13. Adafruit_MPU6050 mpu;
14.
15. // --- Flight Tracking Variables ---
16. float pitch = 0.0f;
17. float roll = 0.0f;
18. const float alpha = 0.96f; // How much we trust the Gyroscope vs Accelerometer
19. uint32_t last_time = 0;
20.
21. void setup() {
22.   Serial.begin(115200);
23.   delay(1000);
24.
25.   // 1. Start the shared I2C Bus on our chosen pins
26.   Wire.begin(I2C_SDA, I2C_SCL, 400000U);
27.
28.   // 2. Connect to the Motion Sensor
29.   if (!mpu.begin()) {
30.     Serial.println("Sensor not found. Check your wiring!");
31.     while (1) { delay(10); } // Stop here if sensor is missing
32.   }
33.
34.   // Set the sensitivity of the sensor
35.   mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
36.   mpu.setGyroRange(MPU6050_RANGE_250_DEG);
37.   mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
38.
39.   // 3. Connect to the Screen
40.   oled.begin();
41.   last_time = micros();
42. }
43.
44. void loop() {

```

```

45. // 1. Calculate how much time has passed since the last loop
46. uint32_t current_time = micros();
47. float dt = (float)(current_time - last_time) / 1000000.0f;
48. last_time = current_time;
49.
50. if (dt <= 0.0f || dt > 0.5f) dt = 0.01f;
51.
52. // 2. Grab the newest data from the sensor
53. sensors_event_t a, g, temp;
54. mpu.getEvent(&a, &g, &temp);
55.
56. // 3. Math: Calculate the raw tilt from gravity
57. float accel_pitch = atan2(-a.acceleration.x, sqrt(a.acceleration.y * a.acceleration.y + a.acceleration.z *
a.acceleration.z)) * RAD_TO_DEG;
58. float accel_roll = atan2(a.acceleration.y, a.acceleration.z) * RAD_TO_DEG;
59.
60. float gyro_pitch_rate = g.gyro.y * RAD_TO_DEG;
61. float gyro_roll_rate = g.gyro.x * RAD_TO_DEG;
62.
63. // 4. Sensor Fusion: Blend the smooth Gyro data with the stable Accelerometer data
64. pitch = alpha * (pitch + (gyro_pitch_rate * dt)) + (1.0f - alpha) * accel_pitch;
65. roll = alpha * (roll + (gyro_roll_rate * dt)) + (1.0f - alpha) * accel_roll;
66.
67. // 5. Start drawing the graphics
68. oled.clearBuffer();
69.
70. // Draw the fixed airplane crosshair in the center of the screen
71. oled.setDrawColor(1);
72. oled.drawCircle(64, 32, 5);
73. oled.drawLine(46, 32, 12);
74. oled.drawLine(70, 32, 12);
75.
76. // 6. Geometry: Calculate where to draw the tilted horizon line
77. float rad_roll = (roll * DEG_TO_RAD);
78. int y_offset = 32 + (int)(pitch * 1.2f); // Move the line up or down based on pitch
79. int span_length = 55; // How long the line is
80.
81. int x1 = 64 - (int)(span_length * cos(rad_roll));
82. int y1 = y_offset - (int)(span_length * sin(rad_roll));
83. int x2 = 64 + (int)(span_length * cos(rad_roll));
84. int y2 = y_offset + (int)(span_length * sin(rad_roll));
85.
86. // Draw the horizon line
87. oled.drawLine(x1, y1, x2, y2);
88.
89. // Print the exact numbers at the top of the screen
90. oled.setFont(u8g2_font_04b_03_tr);
91. oled.setCursor(2, 10); oled.print("PITCH: "); oled.print(pitch, 1);
92. oled.setCursor(88, 10); oled.print("ROLL: "); oled.print(roll, 1);
93.
94. // Send the finished drawing to the OLED
95. oled.sendBuffer();
96. delay(10);
97. }
98.

```